일본어 STT 교정 · 판단 튜터 시스템 목표 아키텍처

기존 GitHub 구조 대비 상세 설계서

- 기준일: 2026 - 09 - 13
- 현재 GitHub 평가 기준선: `origin/api`, commit `8779e0837124e5e0806b550310c4a466a920ab47`
- 현재 상태: 평가셋 · 검수 자료 · 독립 실행기 · 채점기 · 11개 모델 기준선과 최신 아키텍처 문서를 갖췄으며, 다음 단계는 판단 전용 LoRA 학습 데이터 구축
- 문서 성격: 앞으로 구현할 목표 구조와 단계적 전환안. 목표 구성요소가 현재 구현되었다는 뜻은 아님

1. 경영진 요약

현재 시스템은 Raspberry Pi에서 생성한 STT 텍스트를 FastAPI가 받아 SQLite에 저장하고, 규칙 기반 분류와 네 종류의 로컬 검색 결과를 하나의 프롬프트에 넣어 `localhost:1234`의 고정 LLM을 호출하는 단일 백엔드 구조다. 작은 규모의 기능 검증에는 적합하지만 모델 자체 성능 측정, 세 단계 판단, 문맥 · 학습 수준 전달, 모델 버전 관리, 장애 의미 분리, 운영 관측과 점진 배포를 지원하지 않는다.

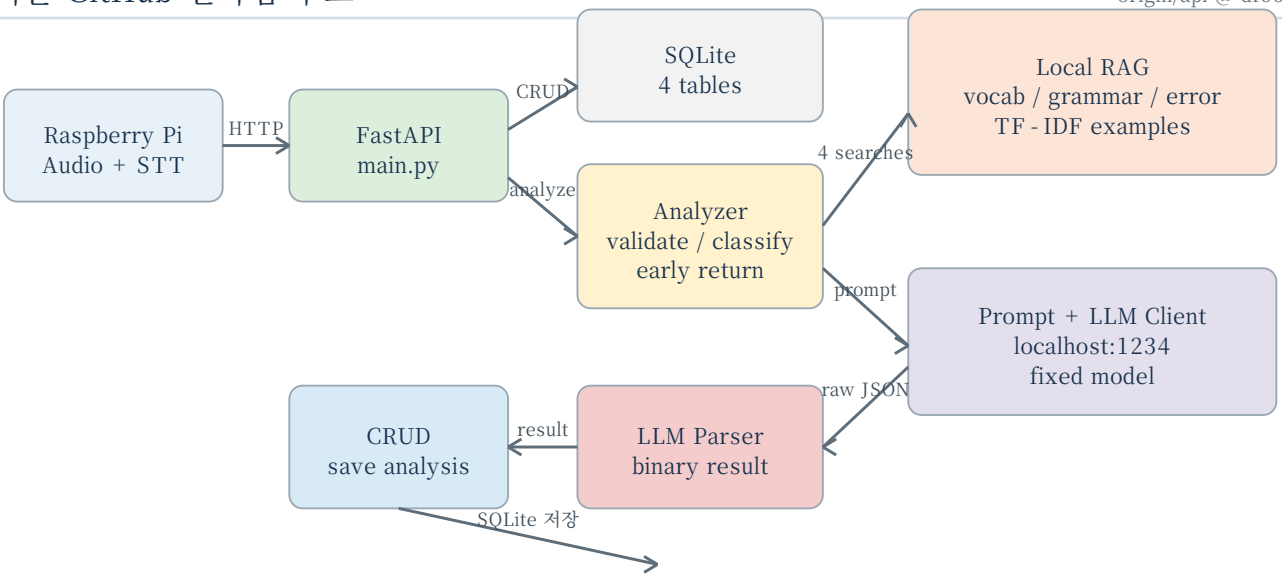
목표 시스템은 실시간 서비스 경로, 오프라인 모델 개발 · 평가 경로, 사람 검수 경로를 분리한다. 실시간 경로에서는 언어학 도구와 검색이 판단을 강제하지 않고 증거를 보조하며, 판단모델은 `acceptable / needs_correction / insufficient_evidence` 중 하나와 최소 교정을 반환한다. 시스템 장애는 판단 보류와 분리한다. 오프라인 경로에서는 정답 비노출 실행기와 고정 시험지로 가장 작은 유망 모델을 선정한 뒤 LoRA를 학습한다.

핵심 결정은 다음과 같다.

- 첫 모델은 학습자에게 점수를 주는 평가모델이 아니라 문법 교정 필요성을 판단하는 모델이다.
- 운영 API보다 독립 실행기 · 채점기와 모델 비교를 먼저 완성한다.
- Sudachi와 RAG는 모델 실패 분석에서 개선 필요성이 확인될 때만 추가한다.
- `で + 行く` 같은 단일 특징으로 오류를 확정하는 하드 게이트를 사용하지 않는다.
- 정상 문장의 과교정을 가장 중요한 위험으로 관리한다.
- 텍스트만으로 학습자의 발음 오류와 STT 전사 오류를 단정하지 않는다.
- 자동 생성 라벨, 사용자 검수, 일본어 전문가 검수를 명확히 구분한다.
- 장기적으로 판단모델과 평가모델을 분리하지만 자동 재학습 · 자동 배포는 초기 범위에서 제외한다.

기존 GitHub 런타임 구조

origin/api @ df66945



핵심 제약: 운영 경로와 모델 평가 결합 · 이진 판단 · 고정 endpoint/model · 문맥/수준 미전달

그림 1. 기존 GitHub 런타임의 실제 호출 · 저장 구조

2. 비교 기준과 현재 상태

2.1 기존 GitHub 기준선

현재 구현 기준은 원격 추적 브랜치 `origin/api`의 commit `8779e0837124e5e0806b550310c4a466a920ab47`이다. 이 커밋에는 API · SQLite · 분석 파이프라인 · RAG 자료와 함께 72문항 평가셋, 독립 실행기 · 채점기, 11개 모델의 재현 요약, 79개 회귀 테스트가 포함되어 있다.

GitHub 기준 주요 디렉터리는 다음과 같다.

```
stt_api/
├── main.py                          # FastAPI 엔드포인트와 분석 실행
├── schemas.py                       # Pydantic 요청 · 응답 모델
├── models.py                        # SQLAlchemy User/Session/Utterance/Analysis
├── crud.py                          # DB 생성 · 조회 · 분석 저장
├── database.py                      # sqlite:///./stt_data.db
├── analyzer/
│   ├── rule_filter.py               # 입력 정규화 · 일본어 여부 확인
│   ├── sentence_classifier.py
│   ├── vocab_retriever.py
│   ├── grammar_retriever.py
│   ├── example_retriever.py
│   ├── error_retriever.py
│   ├── prompt_builder.py
│   ├── llm_client.py                # localhost:1234 OpenAI 호환 호출
│   ├── llm_parser.py
│   └── pipeline.py
├── rag_data/                        # 어휘 · 문법 · 예문 TF-IDF · 오류 패턴
├── scripts/                         # RAG 인덱스 · 단어 DB 생성
├── evaluation/judgment/             # 독립 평가셋 · 실행기 · 채점기 · 결과
└── tests/                           # 전체 79개 회귀 테스트
```

2.2 현재 GitHub에 반영된 평가 기반

`evaluation/judgment/`에는 72문항 평가셋과 검수 자료, 정답 비노출 실행기, 엄격 계약 채점기, 11개 모델의 결과 요약이 반영되어 있다. 72문항은 `user_reviewed`, 검수일은 `2026-09-04`이며 일본어 전문가 인증본은 아니다. 전체 회귀 테스트는 79개다. 평가 경로는 런타임 `analyzer`와 분리되어 있고 아직 운영 API에 연결되지 않았다.

항목	기존 런타임	현재 평가 기반	목표 시스템에서의 역할
최소 판단 평가셋	기존 런타임에는 없음	72문항, 3단계 정답	실행기 스모크 · 회귀 기준
검수 계약	이진 `is_correct` 중심	3단계 판단 · Unicode edit 계약	운영 판단 계약의 기준
검수 문서	기존 런타임에는 없음	Markdown · Excel · PDF	사람 검수 및 변경 추적
데이터 검증기	기존 런타임에는 없음	구조 · edit 복원 · 중복 검사	CI 데이터 게이트
RAG 직접 중복 감사	기존 런타임에는 없음	정규화 직접 일치 검사	누출 위험의 1차 방어
평가 자동화	기존 런타임에는 없음	실행기 · 엄격 채점기 · 비교 보고서 구현	모델 선택과 회귀 측정

루트 `README.md`는 현재 진행 요약과 단계별 기준 문서의 색인으로 관리한다. 상세 평가 수치와 변경 근거는 `PROGRESS.md`와 `evaluation/judgment/FINAL_MODEL_SELECTION.md`를 기준으로 한다.

2.3 현재 검증된 것과 검증되지 않은 것

구분	확인된 상태	해석
Python	프로젝트 venv Python 3.12.3	API 환경 정합성 확보
코드 회귀	79개 통과	코드 · 데이터 구조 검증이며 언어 정확도 인증은 아님
RAG 예문	3,003개 TF-IDF 자료 로딩 · 검색 확인	검색 동작만 확인
Swagger	8001에서 과거 HTTP 200 확인	상시 서비스 구성은 아님

구분	확인된 상태	해석
실제 LLM	RTX A6000에서 11개 후보 평가 완료	모두 잠정 운영 목표 미달
기존 LoRA	Qwen3 4B용 어댑터까지 평가 완료	엄격 계약 0/72로 판단모델 재사용 제외
일본어 전문가 검수	미실시	사용자 검수와 구분해야 함

3. 기존 GitHub 시스템 아키텍처

3.1 런타임 흐름

1. Raspberry Pi 또는 API 사용자가 `/utterances`로 STT 결과를 저장한다.
2. `/utterances/{id}/analyze`가 `processed_text`, `normalized_text`, `stt_text` 순서로 입력을 선택한다.
3. `rule_filter`가 공백을 정규화하고 일본어 문자 포함 여부를 확인한다.
4. `sentence_classifier`가 인사 · 단편 · 일반 문장 등을 분류한다.
5. invalid, greeting, fragment는 LLM 호출 전에 규칙 결과로 조기 반환한다.
6. 그 외 입력은 어휘, 문법, 오류 패턴, 예문 검색을 모두 수행한다.
7. 검색 결과 전체를 `prompt_builder`가 하나의 프롬프트로 합친다.
8. `llm_client`가 고정 주소 `http://localhost:1234/v1/chat/completions`와 고정 모델명 `qwen/qwen3-4b-2507`을 호출한다.
9. `llm_parser`가 JSON을 추출하고 타입을 보정하며 오류 유형을 후처리한다.
10. 결과를 SQLite `analyses` 테이블에 저장하고 API 응답으로 반환한다.

3.2 현재 공개 API

메서드 · 경로	현재 역할	현재 제약
`GET /health`	API 프로세스 상태	DB · RAG · LLM 준비 상태를 구분하지 않음
`POST /users`	사용자 생성	인증 · 중복 정책 · 수준 enum 검증 없음
`POST /sessions`	회화 세션 생성	존재하지 않는 user 참조 정책 미흡
`GET /users/{id}/sessions`	사용자 세션 조회	사용자 존재 여부와 빈 결과 구분 미흡
`POST /utterances`	STT 발화 저장	session 참조 검증과 idempotency 없음
`GET /utterances/{id}`	발화 조회	권한 모델 없음
`PATCH /utterances/{id}/preprocess`	전처리 결과 갱신	처리 단계 · 버전 이력 없음
`POST /utterances/{id}/analyze`	분석 실행 · 저장	동기 호출, 고정 모델, 세 단계 판단 미지원
`GET /utterances/{id}/analysis`	분석 이력 조회	문자열 JSON 필드와 계약 버전 부재
`GET /sessions/{id}/utterances`	세션 발화 조회	페이지네이션 · 권한 · 세션 존재 검증 없음

3.3 현재 데이터 구조

- `User`: 이름, 수준, 생성 시각
- `Session`: 선택적 사용자 참조, 주제, 설명, 생성 시각
- `Utterance`: 오디오 경로, STT 텍스트, 전처리 텍스트, 오디오 메타데이터
- `Analysis`: 문장 분류, 이진 정오답, 교정문, 오류 유형 문자열, RAG 문맥 문자열, 모델명, 신뢰도

배열형 `candidate_types`, `error_types`, `tokens`는 SQLite Text에 JSON 문자열로 저장한다. `create_all`로 테이블을 생성하며 스키마 마이그레이션 체계는 없다. SQLite 외래키 선언은 존재하지만 연결별 강제 활성화와 상위 레코드 사전 검증이 충분하지 않다.

3.4 현재 구조의 장점

- 구조가 단순해 기능 추적과 로컬 테스트가 쉽다.
- FastAPI, SQLAlchemy, OpenAI 호환 LLM 호출이라는 교체 가능한 기본 접점을 갖는다.
- RAG 자료가 로컬에 있어 외부 검색 서비스 없이 동작할 수 있다.
- 잘못된 빈 LLM 응답을 정답으로 저장하던 문제와 혼합 인사 조기 반환 문제는 수정되었다.
- 예문 검색 인덱스와 테스트 기반이 이미 존재한다.

3.5 현재 구조의 핵심 한계

- 운영 파이프라인과 모델 평가 경로가 결합되어 모델 자체 정확도를 측정할 수 없다.
- `is_correct` 이진 값으로는 근거 부족과 시스템 실패를 표현할 수 없다.
- 사용자 수준과 대화 문맥이 실제 판단 프롬프트에 연결되지 않는다.
- 규칙 조기 반환이 모델 판단을 우회하며 단편 발화를 곧바로 오답으로 만들 수 있다.
- 오류 유형 후처리가 일부 복합 오류 정보를 제거할 수 있다.
- 모델 주소 · 이름 · 생성 설정이 코드에 고정되어 있다.
- 모델 · 어댑터 · 프롬프트 · 데이터 리비전의 실행 이력이 없다.
- 검색 근거의 출처 · 버전 · 신뢰도 · 사용 조건이 응답과 감사 로그에 연결되지 않는다.
- 재시도, 회로 차단, idempotency, 큐, 부하 제어, readiness가 없다.
- 인증 · 권한 · 감사 · 개인정보 · 데이터 보존 정책이 없다.

4. 목표 전체 시스템 아키텍처

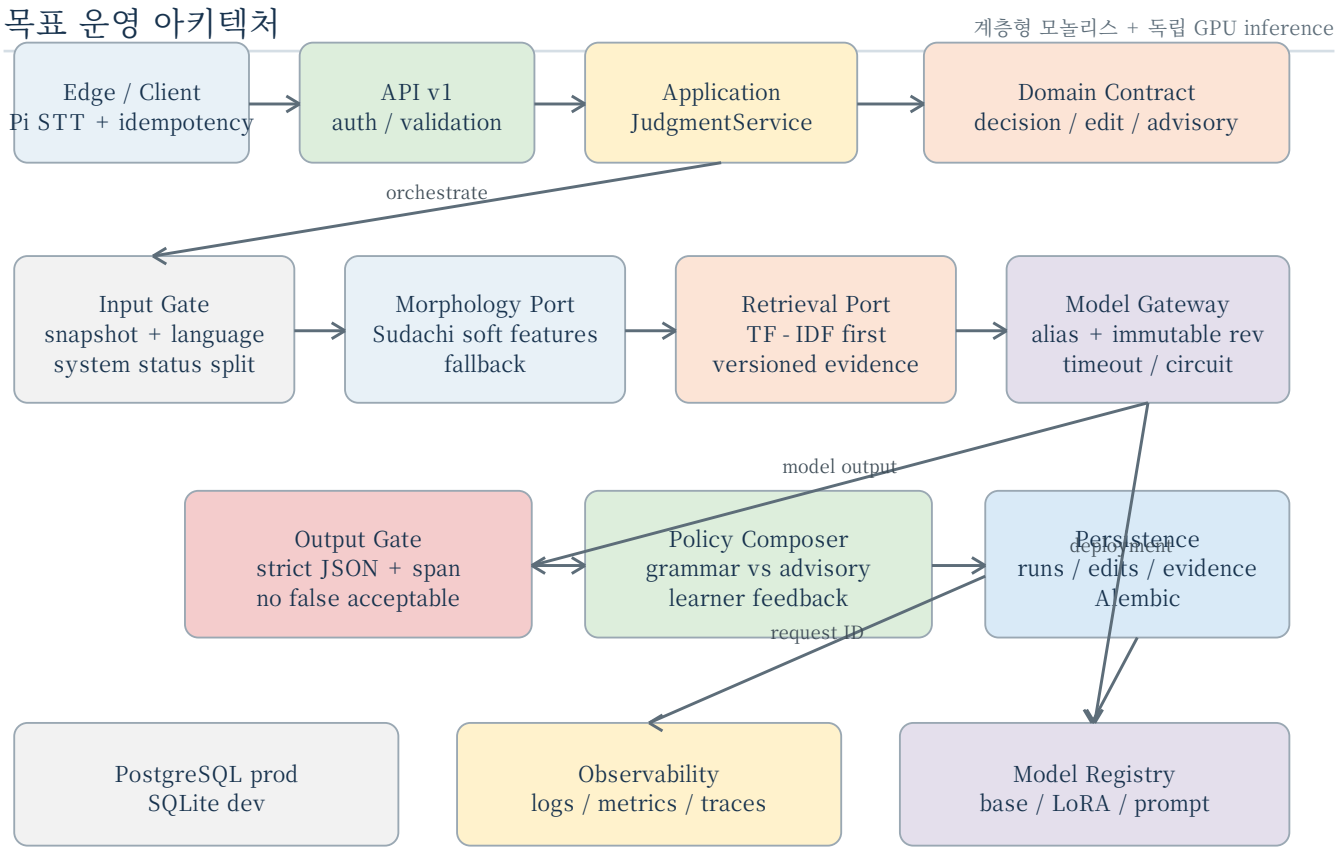


그림 2. 향후 운영 런타임의 책임 분리 구조

4.1 설계 원칙

목표 구조는 모놀리스를 즉시 마이크로서비스로 분해하지 않는다. FastAPI 애플리케이션 내부를 명확한 계층으로 나누고, GPU 모델 서버와 오프라인 ML 파이프라인만 프로세스 경계로 분리한다. 트래픽과 조직 규모가 실제로 필요해질 때 서비스 분리를 선택한다.

계층 의존 방향은 `API → Application Service → Domain → Port`로 고정하고, DB · 검색 · LLM은 Port를 구현하는 Adapter로 둔다. 도메인 판단 규칙은 FastAPI, SQLAlchemy, 특정 모델 서버에 직접 의존하지 않는다.

4.2 Edge/STT 계층

- Raspberry Pi는 녹음, 노이즈 전처리, STT 실행, 기본 오디오 메타데이터 생성까지 담당한다.
- 서버에는 원본 오디오 경로 또는 업로드 참조, STT 텍스트, STT 모델 · 버전, 언어, 신뢰 구간을 전달한다.
- 네트워크 재전송에 대비해 발화 단위 idempotency key를 사용한다.
- 장치 시계가 부정확할 수 있으므로 장치 시각과 서버 수신 시각을 분리해 저장한다.
- 원본 오디오는 DB BLOB가 아니라 접근 통제된 파일 또는 오브젝트 저장소에 둔다.

4.3 API 계층

- `/api/v1` 버전을 사용하고 Pydantic 계약으로 요청 · 응답을 고정한다.
- liveness, readiness, dependency 상태를 분리한다.
- 인증 주체, 세션 소유권, rate limit, request ID를 공통 미들웨어에서 처리한다.

- 동기 판단은 짧은 발화용으로 유지하고, 장문 · 배치 분석은 비동기 job API로 분리한다.
- Swagger에는 운영 기본 모델 alias만 노출하고 임의의 로컬 경로나 모델 ID 입력은 허용하지 않는다.

4.4 Application Service 계층

- `UtteranceService`: 사용자 · 세션 참조 검증, 발화 저장, 전처리 버전 관리
- `JudgmentService`: 입력 스냅샷 작성, 언어학 특징 · 검색 · 모델 호출 조율, 출력 검증, 정책 합성
- `FeedbackService`: 문법 교정과 표기 · 문체 · 문맥 · 대화 복구 조언을 분리해 학습자 응답 생성
- `ModelRoutingService`: 배포 alias를 실제 베이스 모델 · LoRA · 양자화 리비전에 매핑
- `AuditService`: 데이터 · 프롬프트 · 모델 · 검색 근거 · 지연시간 · 오류 상태 기록

4.5 Domain 계층

판단모델의 핵심 계약은 다음 세 상태다.

- `acceptable`: 문법 구조를 고칠 필요가 없음
- `needs_correction`: 의미를 보존하는 최소 문법 교정 근거가 명확함
- `insufficient_evidence`: 발화 누락 · 잡음 · 지원 범위 밖 언어 등으로 문법 판단 근거가 부족함

표기, 발음 귀책, 문체, 질문 관련성, 번역, 대화 복구는 문법 판단과 별도 advisory로 관리한다. `acceptable`은 모든 부가 축이 정상이라는 뜻이 아니다.

4.6 Linguistic Assistance 계층

- 입력 정규화는 원문을 덮어쓰지 않고 정규화본과 변경 내역을 함께 만든다.
- SudachiPy는 형태소 표면형, 기본형, 품사, 활용, 문자 위치를 관찰 정보로 제공한다.
- 형태소 분석 실패 또는 미등록어가 있어도 모델 경로를 막지 않는다.
- 품사 특징은 검색 후보 우선순위와 프롬프트 근거에 사용하지만 정답을 확정하지 않는다.
- 복수 조사 · 동사 · 절 관계를 배열로 유지하고 단일 `particle/verb` 메타데이터로 축약하지 않는다.

4.7 Retrieval 계층

- 1차 구현은 기존 TF - IDF · 키워드 검색을 Adapter로 감싼다.
- 문법 규칙, 정상 반례, 오류 예시, 학습자 수준, 출처 · 라이선스, 버전을 메타데이터로 관리한다.
- 검색 결과는 강제 명령이 아니라 `evidence[]`로 모델에 전달한다.
- 메타데이터 필터 결과가 비거나 형태소 분석이 불확실하면 일반 검색으로 fallback한다.
- ChromaDB · Dense Vector는 동일 시험지에서 유의미한 개선이 확인될 때만 도입한다.
- 학습 · 개발 · 시험 · RAG 사이의 직접 일치와 템플릿 계열 누출을 지속 검사한다.

4.8 Model Inference 계층

- GPU 서버는 API 프로세스와 분리된 OpenAI 호환 endpoint를 제공한다.
- 모델 alias는 model registry의 불변 리비전으로 해석한다.
- 11개 기준선 비교 결과 첫 LoRA 베이스는 `Qwen/Qwen3-4B-Instruct-2507`, 보조 스케일 비교군은 `Qwen/Qwen3-8B`, 고용량 정확도 상한 기준선은 `Qwen/Qwen3-14B`로 정했다.
- 기존 일반 튜터 Qwen3 - 4B LoRA는 동일 베이스보다 계약 성능이 악화되어 판단모델에 재사용하지 않는다.
- 텍스트 전용 · 비사고 모드, 고정 프롬프트와 생성 설정으로 비교한다.
- timeout, 동시성 제한, 회로 차단, 메모리 부족 상태를 API에 명시적으로 전달한다.

- 모델 실패 시 원문을 보존할 수는 있지만 `acceptable`로 승인하지 않는다.

4.9 Output Gate와 Policy 계층

- JSON 객체 · 필수 필드 · enum · 문자열 타입을 엄격히 검증한다.
- edit의 start/end는 Python Unicode code point 기준이며 원문의 source와 일치해야 한다.
- edit를 역순 적용했을 때 `corrected_text`가 정확히 재구성되어야 한다.
- `acceptable`과 `insufficient_evidence`는 원문 및 빈 edit를 유지한다.
- `needs_correction`은 적어도 하나의 실질 edit를 가져야 한다.
- 문법 교정과 advisory 예시를 합치지 않는다.
- 시스템 오류는 `model_timeout`, `model_unavailable`, `invalid_model_output` 등 별도 실행 상태로 반환한다.

4.10 Persistence와 Observability 계층

- 개발은 SQLite를 유지하고 운영은 PostgreSQL을 기본으로 한다.
- Alembic으로 스키마 변경을 버전 관리한다.
- 분석 결과는 `JudgmentRun`, `JudgmentEdit`, `Evidence`, `Advisory`, `ModelDeployment` 개념으로 분리한다.
- raw 모델 응답은 접근 제어와 보존 기간을 적용해 저장하며 API 기본 응답에는 노출하지 않는다.
- request ID를 기준으로 API, 검색, 모델 서버, DB 로그를 연결한다.
- 핵심 지표는 지연시간, 오류율, JSON 성공률, 과교정 감사율, 보류율, 모델별 처리량, GPU 메모리다.
- 로그에 원문 전체를 기본 기록하지 않고 식별자 · 해시 · 길이 중심으로 남긴다.

5. 목표 요청 · 응답 계약

5.1 판단 요청

```
{
    "text": "昨日は図書館を本を読みました。",
    "context": "어제 무엇을 했는지 묻는 질문에 대한 답변입니다.",
    "learner_level": "N5",
    "source": "stt",
    "model_alias": "judgment - default",
    "idempotency_key": "device - 7 - session - 12 - utterance - 31"
}
```

- `text`만 필수다.
- `context`는 v0.1에서 문자열 또는 null로 유지한다.
- `learner_level`은 설명 난이도와 검색 범위에 사용하며 고급 표현을 자동 오답 처리하는 기준이 아니다.
- `model_alias`는 허용 목록만 받고 파일 경로나 임의 모델 ID를 허용하지 않는다.
- 저장된 발화를 분석할 때는 API가 발화 · 세션 정보에서 동일 요청 스냅샷을 구성한다.

5.2 판단 응답

```
{
    "request_id": "01J...",
    "execution_status": "ok",
    "decision": "needs_correction",
    "corrected_text": "昨日は図書館で本を読みました。",
    "edits": [
        {
            "start": 7,
            "end": 8,
            "source": "を",
            "replacement": "で",
            "error_type": "particle_misuse"
        }
    ],
    "reason_ko": "책을 읽는 동작이 일어난 장소이므로 図書館 뒤에는 で를 사용합니다.",
    "advisories": [],
    "evidence": [
        {
            "source_id": "grammar - particle - place - action - v1",
            "kind": "grammar_rule",
            "score": 0.82
        }
    ],
    "model": {
        "alias": "judgment - default",
        "base": "Qwen/...",
        "adapter": "jp - judgment - r1",
        "prompt_version": "judgment - v1"
    },
    "parse_success": true,
    "latency_ms": 412
}
```

`execution_status != ok`이면 문법 `decision`을 생성하지 않는다. 모델 timeout을 `insufficient_evidence`로 위장하지 않으며, 잘못된 응답을 원문과 동일하다는 이유로 `acceptable` 처리하지 않는다.

5.3 Advisory 예시

type	적용 사례	문법 corrected_text 처리
`orthography`	STT의 わたしわ → 쓰기 표기 후보 わたしは	원음 없이는 문법 교정에 강제 반영하지 않음
`register`	면접에서 보통체 · 정중체 통일 권고	문법 허용과 별도로 제공
`context`	문법은 맞지만 질문에 동문서답	원문을 임의의 답으로 교체하지 않음
`recovery`	잡음 · 침묵 · 말 끊김	대화 복구문을 gold 교정문으로 사용하지 않음
`translation`	영어 · 한국어 입력	지원 범위 밖 판단 후 번역 예시로만 제공
`pronunciation_review`	원음 확인이 필요한 조사 발음	텍스트만으로 학습자 오류를 단정하지 않음

5.4 호환 전략

- 기존 `/utterances/{id}/analyze`와 `AnalysisResponse`는 실험 기간 유지한다.
- 새 stateless `POST /api/v1/judgments`를 먼저 추가한다.
- 모델 검증 후 저장형 `POST /api/v1/utterances/{id}/judgments`를 연결한다.
- 기존 `is_correct`는 `acceptable=true`, `needs_correction=false`, 나머지 · 실패=null로 제한적으로 파생하되 신규 소비자는 `decision`을 사용한다.
- 호환 필드는 폐기 예정 표시와 사용량 관측 후 다음 major API에서 제거한다.

6. 목표 데이터 모델

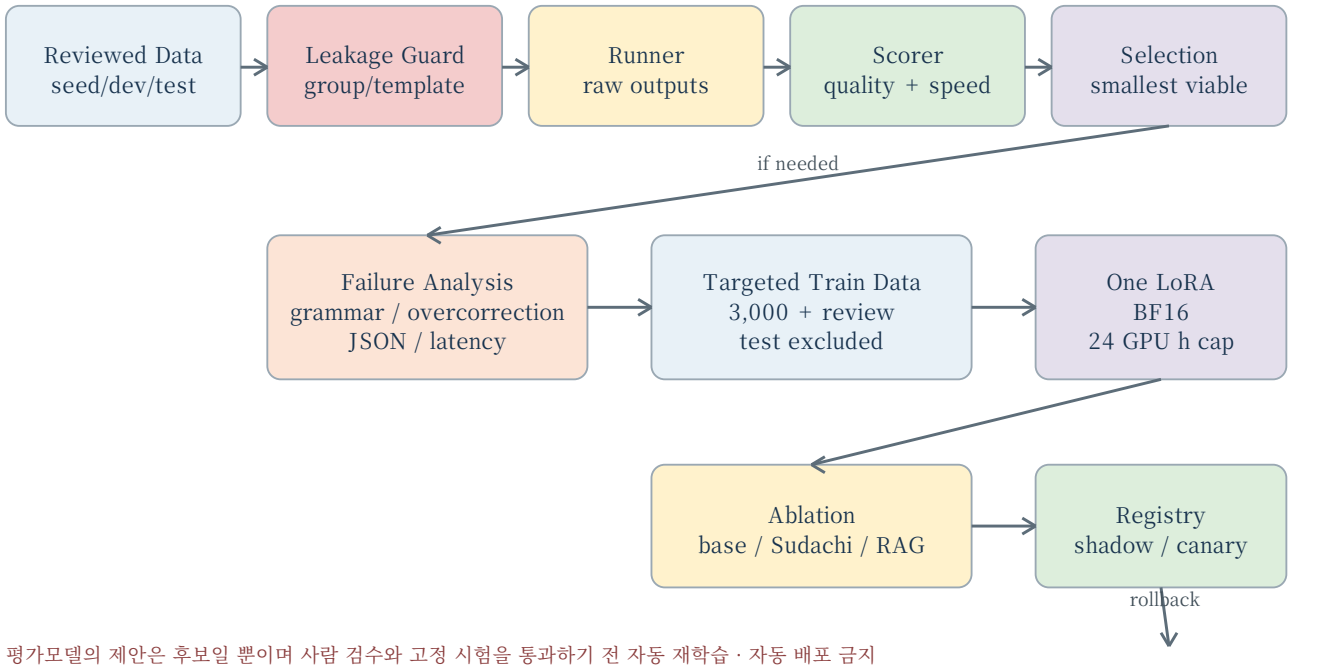
엔터티	핵심 필드	목적
`users`	id, level, consent flags, created_at	사용자 설정과 데이터 사용 동의 분리
`sessions`	id, user_id, topic, context policy	회화 단위 문맥 · 소유권
`utterances`	id, session_id, original STT, normalized snapshot, STT model/version	변경되지 않는 입력 이력
`judgment_runs`	request_id, utterance_id, execution_status, decision, corrected_text, reason, model deployment, prompt version, latency	판단 실행의 중심 감사 레코드
`judgment_edits`	run_id, ordinal, start, end, source, replacement, error_type	복수 · 복합 오류와 위치 보존
`judgment_advisories`	run_id, type, message, example, attribution	문법 외 조언의 분리
`retrieval_evidence`	run_id, source_id, kind, rank, score, corpus version	검색 근거 추적
`model_deployments`	alias, base revision, adapter revision, quantization, prompt, status	재현 가능한 모델 라우팅
`evaluation_runs`	dataset hash, runner version, model revision, metrics, artifact URI	오프라인 성능 기록
`review_events`	item/run id, status, reviewed_at, decision delta	익명 검수와 전문가 검수 상태 구분

운영 스키마에는 개인 검수자 이름을 필수로 저장하지 않는다. 조직상 감사 주체가 필요할 경우 사용자 데이터와 분리된 내부 주체 ID를 별도 보안 정책 아래 저장하며, 현재 익명 사용자 검수본에는 소급 생성하지 않는다.

7. 오프라인 모델 개발 · 평가 아키텍처

오프라인 모델 수명주기

정답 비노출 · 고정 시험지 · 승인 기반 승격



평가모델의 제안은 후보일 뿐이며 사람 검수와 고정 시험을 통과하기 전 자동 재학습 · 자동 배포 금지

그림 3. 데이터에서 승인된 모델 배포까지의 오프라인 수명주기

7.1 데이터 계층

- `seed_eval`: 현재 72문항. 도구 · 계약 스모크와 회귀에 사용한다.
- `dev`: 목표 500문항. 프롬프트 · 모델 · 관문 선택에 사용한다.
- `test`: 목표 1,000문항. 선택 완료 후 한 번 평가하는 고정 시험지다.
- `train`: 목표 3,000문항. dev 실패 유형을 바탕으로 작성하되 test 파생 표현을 넣지 않는다.
- 모든 데이터는 source, license/creation basis, group, template family, review status, hash를 가진다.

7.2 정답 비노출 실행기

- 실행기는 모델에 `text`, `context`, `learner_level`만 보낸다.
- expected, accepted corrections, reason, tags, review는 프롬프트에서 제외한다.
- 모델 원시 응답과 실행 manifest를 먼저 저장한 뒤 별도 채점기가 정답을 결합한다.
- prompt, generation config, package lock, CUDA, GPU, model revision을 기록한다.

7.3 채점 지표

- 과교정률: gold acceptable 중 needs_correction으로 바꾼 비율
- 탐지 정밀도 · 재현율: needs_correction 검출 기준
- 올바른 보류율: gold insufficient_evidence를 동일하게 판단한 비율
- 명확 문항 처리율: gold acceptable/needs_correction을 불필요하게 보류하지 않은 비율
- JSON 성공률: strict contract를 그대로 통과한 비율
- 교정문 일치율: canonical 또는 accepted correction과 일치한 비율
- edit 정확도: 위치 · source · replacement · error type의 일치
- 복합 오류 recall: 한 문장 내 gold 오류를 빠짐없이 탐지한 비율
- 의미 보존 · 문체/표기/문맥 분리: 표본 사람 검수
- 효율: VRAM, cold start, p50/p95 latency, tokens/s

7.4 모델 선택과 LoRA

- Qwen3.5 0.8B · 2B · 4B, Qwen3 4B · 8B · 14B, RakutenAI 1.5B, llm-jp 1.8B · 3.7B, Phi-4 mini와 기존 Qwen3 4B LoRA까지 11개 후보를 평가했다.
- 모든 후보는 72문항과 `judgment-v0.1`, `temperature=0`, `top_p=1`, RAG 미사용 조건으로 비교했다. 런타임 차이가 있는 지연시간은 동일 엔진 모델 사이에서만 직접 비교한다.
- Qwen3-8B가 엄격 계약 49/72 · 판단 정답 39/72로 가장 높고, Qwen3-14B가 원시 판단 60/72 · 허용 교정문 19/27로 내용상 가장 높았으나 11개 모두 잠정 운영 목표를 통과하지 못했다.
- 비용 대비 첫 LoRA 베이스는 Qwen3-4B, 보조 학습 비교군은 Qwen3-8B, 학습된 소형 모델의 상한 비교군은 Qwen3-14B로 확정했다.
- 목표는 과교정률 2% 이하, 정밀도 95% 이상, 재현율 80% 이상, 명확 처리율 85% 이상, JSON 99% 이상이다.
- 첫 판단 전용 LoRA는 Qwen3-4B BF16으로 학습하고 첫 실험 GPU 상한은 24시간이다. 72문항 seed는 개발 진단에만 사용하며 별도 학습 데이터에 포함하지 않는다.
- 상세 수치와 선정 근거는 `evaluation/judgment/FINAL_MODEL_SELECTION.md`를 기준으로 한다.

7.5 향후 평가모델

판단모델이 안정화된 뒤 별도의 평가모델이 판단의 과교정, 누락, 근거 적합성을 검토한다. 평가모델의 제안은 학습 데이터 후보일 뿐 정답이 아니다. 사람 검수 · 누출 검사 · 고정 시험을 통과하기 전 자동 재학습이나 운영 모델 교체에 사용하지 않는다.

8. 기존 구조와 목표 구조의 상세 차이

8.1 제품 · 런타임

영역	기존 GitHub 구조	목표 구조	전환 영향
시스템 경계	FastAPI 단일 앱 + 외부 LLM	계층화 FastAPI + 독립 GPU inference + 오프라인 ML	런타임과 평가 책임 분리
API 버전	루트 경로, 1.0.0 표기	`/api/v1` 명시 버전	구 API 병행 후 단계 폐기
판단 API	저장된 utterance만 분석	stateless judgment + 저장형 judgment	Swagger 직접 시험 가능
판단 상태	bool `is_correct`	3단계 `decision` + execution status	DB · Pydantic 마이그레이션 필요
문맥	저장은 가능하나 모델 미전달	입력 스냅샷에 context 포함	문맥 중복 문항을 구분 가능
학습 수준	User에 문자열 저장만	검증 enum + 설명/RAG 범위에 전달	수준을 오류 판정 하드 규칙으로 사용하지 않음
동기 처리	모든 분석이 요청 안에서 실행	짧은 판단 동기, 배치 · 장문 job 분리	timeout · 부하 제어 개선
조기 반환	greeting/fragment 규칙이 판단 확정	안전 shortcut만 유지, 보류 계약 사용	모델 비교 왜곡 제거
오류 응답	원문 유지와 null 중심	실행 실패 코드와 언어 판단 분리	false acceptable 방지
idempotency	없음	장치 · 발화 · 판단 요청 key	재전송 중복 저장 방지

8.2 모델 · 언어 · 검색

영역	기존 GitHub 구조	목표 구조	전환 영향
모델 선택	코드에 모델명 고정	registry alias → 불변 revision	배포와 재현성 확보
endpoint	localhost:1234 고정	환경 설정 + readiness + routing	개발 · 운영 분리
adapter	응답 필드는 있으나 적용 경로 없음	base/LoRA 호환성 검증 후 명시 로드	Qwen3 LoRA를 Qwen3.5에 오적용 금지
생성 설정	함수 기본값	실행 manifest와 배포 설정 버전화	모델 비교 공정성 확보
출력 계약	이진 JSON + 후처리	3단계 strict JSON + Unicode edits	보류 · 복합 오류 지원
형태소	없음	Sudachi soft feature adapter	관찰 정보이며 판정기가 아님
RAG 실행	유효 일반 문장에 네 검색 모두 실행	query plan과 선택적 evidence	지연 · 노이즈 감소
RAG 방식	TF-IDF + 키워드	기존 방식 우선, vector는 ablation 후	Chroma 선도입 방지
메타데이터	자료별 상이	수준 · 출처 · 라이선스 · 버전 · 반례	근거 감사와 필터 개선
출력 게이트	타입 보정과 일부 휴리스틱	schema · edit 복원 · policy 분리	후처리로 오류를 덮어쓰지 않음

8.3 데이터 · 품질 · 운영

영역	기존 GitHub 구조	목표 구조	전환 영향
평가셋	GitHub 기준 없음	seed/dev/test/train registry	모델 성능을 서비스 코드와 분리
검수 상태	없음	provisional/user/expert 구분	인증 수준 과장 방지
데이터 누출	검사 없음	exact + template/group 분리	시험지 오염 방지
DB	SQLite + create_all	SQLite dev, PostgreSQL prod, Alembic	안전한 스키마 진화
복수 오류	문자열 JSON	정규화된 edit 레코드 또는 JSON column	복합 오류 보존
근거 추적	RAG 문맥 문자열 저장	evidence source/rank/version	설명 가능성과 재현성 향상
관측	기본 로그 수준	metrics, tracing, structured logs	병목 · 장애 · 회귀 탐지
보안	인증 · 권한 없음	인증, 소유권, rate limit, secret 분리	엔터프라이즈 적용 기반
개인정보	명시 정책 없음	최소 수집, 원문 로그 제한, 보존 · 동의	운영 데이터 오용 방지

영역	기존 GitHub 구조	목표 구조	전환 영향
배포	수동 프로세스	health/readiness, shadow, canary, rollback	모델 안전 배포
CI	코드 테스트	코드+계약+데이터+누출+마이그레이션	데이터 변경도 릴리스 게이트화

8.4 저장소 목표 구조

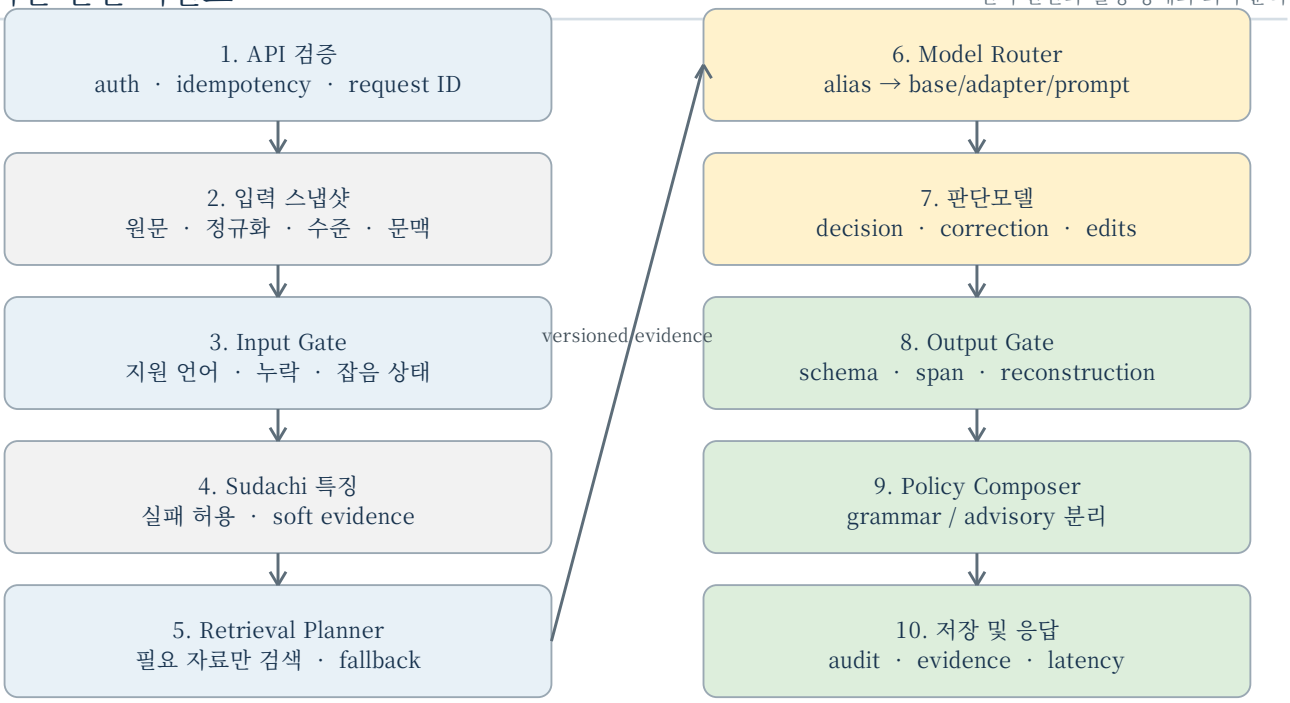
```
stt_api/
├── app/
│   ├── api/v1/                    # users, sessions, utterances, judgments, health
│   ├── application/               # orchestration services and DTOs
│   ├── domain/judgment/           # decision, edit, advisory, invariants
│   ├── ports/                     # model, retrieval, morphology, repository
│   ├── adapters/
│   │   ├── inference/             # OpenAI-compatible model gateway
│   │   ├── retrieval/             # TF-IDF first, optional vector later
│   │   ├── morphology/            # Sudachi adapter
│   │   └── persistence/           # SQLAlchemy repositories
│   ├── observability/             # logging, metrics, tracing
│   └── settings.py                # environment-based validated config
├── migrations/                    # Alembic schema history
├── evaluation/judgment/           # datasets, contract, runner, scorer, reports
├── training/judgment/             # LoRA configs and reproducible manifests
├── rag_data/                      # versioned corpora and metadata manifests
├── tests/
│   ├── unit/
│   ├── integration/
│   ├── contract/
│   └── evaluation/
├── docs/architecture/
└── deploy/                        # service/container config after validation
```

초기 전환에서는 기존 파일을 한 번에 이동하지 않는다. 먼저 새 judgment 도메인과 adapter를 추가하고 기존 endpoint가 이를 호출하도록 전환한 뒤, 테스트가 유지되는 단위로 API · DB 파일을 이동한다.

9. 실시간 판단 데이터 흐름

실시간 판단 시퀀스

언어 판단과 실행 장애의 의미 분리



timeout / unavailable / invalid output → execution_status 실패, 문법 insufficient_evidence와 합치지 않음

그림 4. 한 발화가 최종 판단으로 저장되는 순서

1. API가 인증 · rate limit · idempotency와 요청 타입을 검증한다.
2. 원문, 정규화본, STT 메타데이터, 수준, 문맥을 불변 입력 스냅샷으로 만든다.
3. 입력 게이트가 빈 입력, 지원 범위 밖 언어, 잡음 마커 등 관찰 상태를 만든다.
4. Sudachi adapter가 형태소 특징을 추출하되 실패해도 파이프라인을 계속한다.
5. Retrieval planner가 입력과 특징에 따라 필요한 자료만 검색하고 evidence를 버전과 함께 반환한다.
6. Model router가 배포 alias를 실제 base · adapter · prompt 리비전으로 해석한다.
7. 판단모델이 문법 decision, 최소 corrected text, edits, reason을 생성한다.
8. Output gate가 JSON, enum, Unicode span, edit 재구성, no-op 규칙을 검증한다.
9. Policy composer가 문법 결과와 표기 · 문체 · 문맥 · 복구 advisory를 분리한다.
10. 감사 레코드와 근거 · 모델 버전 · 지연시간을 저장하고 API 응답을 반환한다.

실패 처리 원칙

실패	API 의미	사용자 응답	저장
빈 입력 · 지원 불가 타입	4xx validation	다시 입력 요청	민감 원문 없이 검증 이벤트
모델 timeout	504 또는 비동기 재시도	현재 판단 불가	execution_status=timeout
모델 서버 불가	503	잠시 후 재시도	회로 차단 상태와 배포 ID

실패	API 의미	사용자 응답	저장
잘못된 모델 JSON	502	결과를 승인하지 않음	raw response 제한 저장, parse_success=false
전사 누락 · 잡음	정상 200	insufficient_evidence + 복구 안내	언어 판단으로 저장
형태소 분석 실패	모델 경로 계속	필요 시 advisory 없음	morphology_status=failed
검색 결과 없음	일반 모델 fallback	근거 없는 강제 규칙 없음	empty evidence 기록

10. 보안 · 개인정보 · 엔터프라이즈 운영

- API secret, DB URL, 모델 경로는 환경 기반 secret으로 관리하고 소스에 넣지 않는다.
- 사용자 · 세션 · 발화는 소유권 검사를 적용한다.
- 운영 원문과 오디오는 모델 학습에 자동 사용하지 않는다.
- 학습 활용은 명시적 동의, 비식별화, 보존 기간, 삭제 처리 경로를 갖춘 별도 export로만 수행한다.
- 로그는 request ID, 상태, 해시, 길이, latency 중심이며 원문 전체는 debug 기본값에서 제외한다.
- 모델 · 프롬프트 · RAG 변경은 승인된 manifest와 rollback 대상을 가진다.
- dependency lock, SBOM, 취약점 검사, 비밀 탐지, 라이선스 manifest를 CI에 추가한다.
- DB backup/restore 훈련과 모델 artifact 무결성 검사를 운영 준비 조건으로 둔다.
- 장애 시 이전 안정 모델 alias로 되돌릴 수 있어야 한다.

11. 단계별 구현 · 전환 로드맵

단계 0 — 기준점 고정: 완료

- 72문항 평가셋 · 검수 문서 · Excel · PDF를 Git에 반영하고 `judgment-seed-v0.1` 태그로 고정했다.
- 72문항 모두 익명 `user_reviewed`이며 데이터 SHA - 256은 `f681ba026cfd6eeca86ebf531e1ac94ceed06525b43fca2b19729ef86ff0d12b`다.
- 일본어 전문가 검수본이 아니며, 이후 모델 개발을 본 평가셋에 맞추지 않는다.

단계 1 — 독립 실행기 · 채점기: 완료

- 정답 비노출 모델 runner, strict parser, scorer, 비교 보고서와 fake endpoint 회귀 테스트를 구현했다.
- 모델에는 `text`, `context`, `learner_level`만 전달하며 원시 출력과 manifest를 분리 보존한다.
- 운영 `analyzer`를 호출하지 않고 72문항을 실행 · 채점하며 데이터 · 프롬프트 · 결과 SHA - 256을 검증한다.

단계 2 — 베이스 모델 비교: 완료

- API venv와 분리한 Python 3.12 모델 환경에서 11개 후보를 실행했다.
- 모든 후보의 모델 revision · 가중치 해시 · 생성 설정 · 품질 · 지연시간을 기록했다.
- 첫 LoRA 베이스는 Qwen3 - 4B, 보조 비교군은 Qwen3 - 8B, 고용량 상한은 Qwen3 - 14B로 정했다.
- 72문항은 앞으로 개발 진단용 seed로만 사용하고, 학습 · 개발 · 미사용 시험 데이터는 별도로 구축한다.

단계 3 — 판단 특화 LoRA: 다음 작업

- dev 실패를 기준으로 train 3,000을 구성하고 선택된 한 모델만 학습한다.
- 학습 전후를 동일 scorer로 비교한다.
- 완료 기준: 데이터 · 코드 · 모델 · adapter · prompt manifest와 재현 가능한 결과 보존.

단계 4 — 언어학 · RAG ablation

- 모델 단독, +Sudachi, +RAG, +Sudachi+RAG를 dev에서 비교한다.
- 개선된 조합만 고정 test에서 한 번 확인한다.
- 완료 기준: 과교정 악화 없이 통계 · 사례 양쪽에서 개선이 확인된 구성만 채택.

단계 5 — 신규 judgment API와 DB 마이그레이션

- domain contract, adapters, stateless `/api/v1/judgments`를 추가한다.
- Alembic으로 judgment run · edit · advisory · evidence 저장 구조를 추가한다.
- 기존 분석 endpoint는 호환 adapter로 유지한다.
- 완료 기준: Swagger에서 정상 · 교정 · 보류 · timeout · invalid JSON을 재현하고 DB 결과가 일치.

단계 6 — Shadow와 제한 배포

- 기존 응답을 유지하면서 새 판단을 사용자에게 노출하지 않는 shadow 모드로 수집한다.
- 사용자 동의가 있는 표본만 수동 검수하고 과교정 · 오류율 · latency를 비교한다.
- 기준 충족 시 제한 사용자 canary, 이후 기본 alias를 전환한다.
- 완료 기준: rollback, readiness, alert, 데이터 삭제 경로까지 운영 점검 완료.

단계 7 — 장치 최적화와 두 모델 체계

- 서버형 확정 후 양자화별 품질 · 속도를 재측정한다.

- 실제 4GB · 8GB 장치가 정해진 뒤 device model을 별도 선정한다.
- 판단모델 안정화 후 평가모델과 사람 검수 기반 개선 루프를 추가한다.
- 완료 기준: 자동 모델 교체 없이 승인된 artifact만 registry alias로 승격.

12. 테스트 전략과 승인 게이트

계층	필수 테스트	승인 조건
Domain	decision/edit/advisory 불변식	모든 경계 사례 통과
Parser	누락 필드, NaN, 잘못된 enum, 코드펜스, span	실패를 acceptable로 변환하지 않음
Retrieval	빈 결과, fallback, 중복, 출처 · 버전	hard gate 없이 결정적 결과
Model gateway	timeout, 429/5xx, 잘못된 JSON, adapter metadata	재시도 · 회로 차단 · 상태 분리
Repository	FK, transaction, migration upgrade/downgrade	고아 레코드와 부분 저장 없음
API contract	4xx/5xx, idempotency, old/new compatibility	OpenAPI snapshot과 일치
Evaluation	label leakage, accepted correction, metric fixtures	정답이 모델 요청에 포함되지 않음
End - to - end	Pi 입력 모사 → judgment → 저장 · 조회	request ID와 버전 추적 가능
Performance	concurrency, p50/p95, GPU OOM	설정된 SLO 안에서 안정
Security	auth, ownership, secret/log scan	타 사용자 데이터 접근 불가

모델 품질 목표는 초기 기준이며 실제 표본 수와 신뢰구간을 함께 보고한다. 기준 미달 결과를 숨기거나 시험지를 보면서 모델을 수정하지 않는다.

13. 주요 위험과 완화책

위험	영향	완화
72문항 과적합	성능 과대평가	seed는 스모크로 제한, 독립 dev/test 구축
자동 라벨 오류	잘못된 LoRA 학습	review status 분리, 중요 표본 전문가 검수
과교정	학습자 신뢰 저하	acceptable 비율 확보, 과교정률 우선 gate
STT와 학습자 귀책 혼동	부당한 피드백	원음 없으면 attribution unknown advisory
형태소 하드 게이트 오판	정상 문장 차단	soft feature와 fallback, 정상 반례 포함
RAG 누출	평가 무효	exact/template/group audit와 corpus versioning
모델 · LoRA 비호환	실행 실패 · 성능 저하	base revision 검증, registry compatibility rule
시스템 오류의 정답 위장	잘못된 저장	execution status와 decision 분리
SQLite 운영 한계	동시성 · 마이그레이션 위험	SQLite dev, PostgreSQL prod, Alembic
원문 로그 노출	개인정보 위험	최소 로그, 접근 통제, 보존 · 삭제 정책
장치 성능 추정 오류	배포 실패	실제 대상 장치 확정 후 별도 benchmark

14. 최종 목표 상태

최종 시스템은 Raspberry Pi STT 입력을 받아 학습자 수준과 대화 문맥을 포함한 판단 요청을 만들고, 검증된 최소 크기 모델과 선택적 언어학 · 검색 근거를 이용해 문법 판단과 최소 교정을 생성한다. 문법, 표기, 발음 귀책, 문체, 문맥, 복구, 번역을 분리하며 모든 결과는 모델 · 프롬프트 · 데이터 · 검색 버전과 함께 추적된다.

기존 시스템의 장점인 단순한 FastAPI · 로컬 RAG · OpenAI 호환 모델 접점은 유지한다. 달라지는 핵심은 모델을 먼저 독립 평가하는 구조, 세 단계 판단 계약, 언어학 도구를 보조 증거로 다루는 정책, 모델 · 데이터 수명주기와 운영 감사의 분리다.

이 문서의 목표 구조는 단계별 승인 게이트를 통과하며 구현한다. 특히 모델 비교 전 운영 API 계약을 성급하게 확장하거나, 실측 없이 ChromaDB · 대형 모델 · 하드 메타데이터 게이트를 필수 요소로 확정하지 않는다.

부록 A. 문서 근거

- `origin/api` 평가 기준선 commit `8779e0837124e5e0806b550310c4a466a920ab47`의 Git tree
- `main.py`, `schemas.py`, `models.py`, `crud.py`, `database.py`
- `analyzer/`의 입력 검증 · 분류 · 검색 · 프롬프트 · LLM · 파서 · 파이프라인
- `rag_data/`의 어휘 · 문법 · 예문 TF - IDF · 오류 패턴 자료
- 현재 진행 기준 `PROGRESS.md`
- `evaluation/judgment/README.md`의 평가셋 · 실행기 · 채점기 사용법
- `evaluation/judgment/FINAL_MODEL_SELECTION.md`의 11개 모델 결과와 최종 후보 선정 근거
- `evaluation/judgment/`의 72문항 계약 · 검수 자료 · 모델별 재현 요약
- 현재 회귀 테스트 결과: 79개 통과

부록 B. 명시적 비범위

- 베이스 모델 다운로드와 오프라인 기준선 평가는 완료했지만, 판단 전용 LoRA 학습과 운영 배포가 완료되었다고 주장하지 않는다.
- 72문항 사용자 검수는 일본어 전문가 인증이 아니다.
- SudachiPy는 의존 관계와 문법 정답을 100% 보장하는 도구로 취급하지 않는다.
- ChromaDB, Graph DB, 별도 평가모델은 초기 필수 구성요소가 아니다.
- 실제 라즈베리파이 메모리 · 지연 성능은 대상 장치에서 별도로 검증한다.
- 자동 재학습과 자동 운영 모델 교체는 포함하지 않는다.